AI-Powered Performance Marketing Optimization Agent

A closed-loop, agentic AI system that diagnoses and corrects underperformance in an advertising account — autonomously, and under enforced guardrails

Working prototype | Simulated advertising environment | Individual project | Python, stdlib only | Zero API cost

01 What this is

Ad platforms like Meta and Google already report *what* happened to a campaign — CTR, CPA, ROAS. They do not say *why* performance moved, *which* of several plausible causes is responsible, *what* to change, or *whether the change worked*. That diagnostic loop is normally manual, slow, and inconsistent — and acting on the wrong diagnosis wastes money twice: the spend itself, and the disruption caused by an unnecessary change.

This project is an autonomous agent that performs that diagnostic loop end-to-end inside a **controlled, simulated advertising account** I built specifically for this purpose. The agent inspects account performance, forms and weighs competing hypotheses, takes real actions through a validated tool interface, observes the measured consequences, and decides whether to act again or stop — all under hard, code-enforced guardrails the model cannot override.

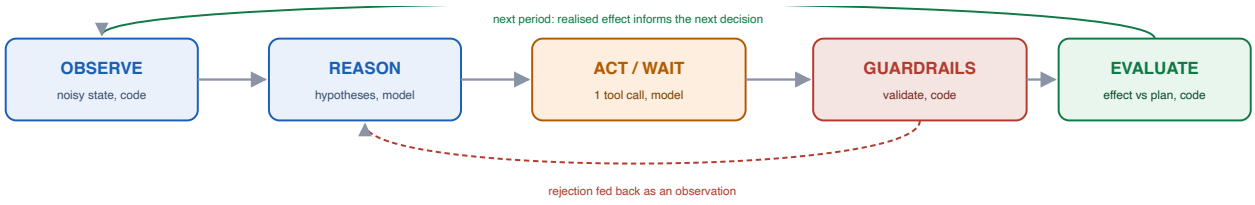
4	8	35	98	$0
DEFECT SCENARIOS + 1 RESTRAINT TEST	TOOLS THE AGENT CAN CALL	GUARDRAIL RULES OUTSIDE THE MODEL	AUTOMATED TESTS IN THE REPO	API SPEND ACROSS THE WHOLE PROJECT

02 Why this is genuinely agentic — not a prompt

It would be easy to point an LLM at a spreadsheet and ask for "recommendations." That is not what this is. Three things make this system agentic rather than a wrapper:

- **The model chooses the sequence.** There is no fixed pipeline. On every turn the agent decides which tool to call, on which entity, with what arguments — and whether to investigate further, act, wait, or stop. Two runs on the same scenario take structurally different paths.
- **Its actions change the world, and it is held to the consequences.** Every consequential action requires the agent to state an *expected effect* in advance (which metric, which direction, by how much, within how many periods). The system then measures the *realised* effect independently and marks it confirmed, partial, or contradicted — the model's own prediction is never treated as ground truth.
- **It operates inside guardrails it does not control.** A deterministic validation layer — entirely outside the model — checks every action before it can execute. An illegal or unjustified action is rejected with a machine-readable reason that is fed back to the agent as a new observation, so it has to adapt rather than being blocked silently.

The decision loop



Blue and orange steps are where the model reasons or acts; red and green steps run entirely in deterministic code and cannot be influenced by the model's output.

03 The environment the agent operates in

Rather than connect to a live Meta or Google Ads account — which would make it impossible to safely test whether an autonomous agent's decisions actually help — I built a **simulated advertising account** with hidden ground truth the agent can never read directly: 3 campaigns, 7 ad sets and 14 creatives, with 12 periods of historical data before the agent's 6-period decision window begins.

The simulator models mechanisms real ad accounts exhibit: diminishing returns from over-spending past an ad set's efficient capacity, audience saturation from over-frequency on small audiences, creative fatigue, a temporary efficiency penalty ("learning phase") after any structural change, sampling noise, and a realistic reporting delay where a share of conversions only appear the following period. The agent only ever sees noisy, partially delayed samples of this — never the underlying truth — so it has to genuinely infer the cause from incomplete evidence.

Four scripted account conditions, plus one deliberate trap. Three scenarios each contain one specific, hidden root cause (a fatigued creative, an over-saturated audience, a misallocated budget). A fourth contains *no real problem at all* — every metric is healthy and any apparent swing is sampling noise. This tests restraint: a good agent has to be able to conclude that no action is the right action, even with every tool still available to it.

04 What the agent can actually do

The agent's entire interface to the account is eight tools. It chooses which one to call, in what order, and how many times.

TOOL	KIND	WHAT IT DOES
`get_account_snapshot`	READ	Current structure and performance of every campaign / ad set / creative
`get_timeseries`	READ	Period-by-period history for one entity, with fitted trends
`compare_segments`	READ	Statistically tests whether an apparent difference between two entities is real, or just noise
`set_budget`	ACTION	Change one ad set's daily budget (raise or cut, within a hard step limit)
`set_status`	ACTION	Pause or resume an ad set or a single creative
`launch_creative_test`	ACTION	Add a new, untested creative to an ad set
`advance_period`	CONTROL	Let one period elapse and observe the outcome — this is also how the agent waits
`finish`	CONTROL	End the run when no further action has positive expected value

All arithmetic — CTR, CPA, ROAS, significance tests, trend fitting — is computed in deterministic code before the model ever sees it. The model interprets pre-computed, verified numbers; it never calculates a metric itself.

05 Guardrails: what the model cannot do

Every action passes through a validation layer, entirely outside the model, before it can touch the environment. 35 distinct rules cover a maximum change per action, per-account and per-campaign budget ceilings, a minimum amount of data before a pause or cut is allowed, a hard cap on structural changes per run, and a "pacing guard" that refuses a budget increase the remaining budget cannot fund. The agent must also state, for every consequential action, why it chose that level of intervention over the alternative, and whether the action resembles one it already tried and saw fail — both are checked mechanically, not just asked for. A rejected action is never silently dropped: the reason comes back as a new observation the agent has to respond to on its next turn.

Evidence: a real decision trace from the agent's most recent run

```
detail  : ready (model=sonnet, cost reported by CLI: 0.027691 4)
sailesh@Saileshs-MacBook-Air performance-marketing-agent % python3 -m adops run --scenario S1_creative_fatigue --seed 1 --policy llm --report
llm backend: claude_cli (ready (model=sonnet, cost reported by CLI: 0.003916400000000004))
scenario S1_creative_fatigue  seed 1  policy llm
state before the run:
period 0/6   target CPA 37.00   budget 20563   spend 0   remaining 20563
CPA to date: None   conversions: 0.0   pacing vs plan: None%
ad_set           budget    spend     conv      CPA     CTR%      CPM   freq       data
as_interior        60.0     1260     20.2    62.22    1.124    11.83   1.06   adequate
as_broad          150.0     3150     56.8    55.51    1.027    10.06   1.13   adequate
as_seas_broad      43.0      903     19.8    45.72    1.277    11.19   1.05   adequate
as_lal             95.0     1995     52.5    38.00    1.262    12.39   1.11   adequate
as_seas_gift       37.0      777     24.0    32.38    1.209    12.64   1.07   adequate
as_rt30            50.0     1050     45.5    23.08    1.473    18.41   1.77   adequate
as_cart            45.0      945     50.2    18.81    1.309    22.12   2.14   adequate

decision trace:
  [01] p0 ok  compare_segments       investigate
  [02] p0 ok  compare_segments       investigate
  [03] p0 ok  get_timeseries         investigate
  [04] p0 ok  set_status             act
  [05] p1 ok  advance_period         wait
  [06] p1 ok  get_timeseries         investigate
  [07] p2 ok  advance_period         wait
  [08] p2 ok  compare_segments       investigate
  [09] p2 ok  compare_segments       investigate
  [10] p2 ok  get_timeseries         investigate
  [11] p2 ok  get_timeseries         investigate
  [12] p2 REJ set_status             campaign_floor_breach
  [13] p2 ok  set_budget             act
  [14] p2 ok  compare_segments       investigate
  [15] p2 ok  get_timeseries         investigate
  [16] p2 ok  set_budget             act
  [17] p3 ok  advance_period         wait
  [18] p3 ok  compare_segments       investigate
  [19] p3 ok  get_timeseries         investigate
  [20] p3 REJ set_budget             pattern_check_missing
```

Unedited terminal output from `python3 -m adops run --scenario S1_creative_fatigue --seed 1 --policy llm --report`, running on my own Claude subscription at zero marginal API cost. The account state table is computed entirely by the platform; the trace below it is the sequence the model chose for itself — a mix of investigation calls and actions, including one action rejected live by the guardrail layer (REJ), which the agent then had to respond to on its next turn.

06 How the system is evaluated

Every run is seeded, so the identical (scenario, seed, policy) triple always produces the identical result — the evaluation is reproducible on demand, not a one-off demo. I compare the agent against two deterministic reference policies that share the exact same runtime, tools and guardrails, so the comparison is apples-to-apples:

- **Do-nothing** — the lower bound. Never intervenes.
- **Rule-based auto-optimiser** — a reasonable, non-strawman deterministic policy that checks statistical significance before acting and won't touch the same entity twice in quick succession — the kind of automation a platform might ship out of the box.
- **Oracle** (evaluation-only) — applies the scenario's actual known-correct fix directly. It exists purely to prove each scenario really does reward the correct diagnosis, before trusting any comparison against it.

> **The restraint scenario validates the whole harness.** On the no-real-problem scenario, across a 4-seed matrix, the do-nothing policy and the oracle score **identically** (553.0 average) — correctly proving that no action beats no action when nothing is actually wrong. The rule-based baseline intervenes anyway and scores **9.0% worse** than doing nothing. This is the standard the agent has to clear, and it is not a low bar: the deterministic baseline actively loses to inaction on 2 of the 4 scenarios tested.

Objective score (conversions, penalised for missing the CPA target or exceeding budget) — averaged across 4 fixed seeds per scenario:

SCENARIO	DO-NOTHING	RULE BASELINE	ORACLE (CEILING)	WHAT IT PROVES
Creative fatigue	521.8	490.9 (–5.9%)	599.5 (+14.9%)	Correct diagnosis is worth acting on
Audience saturation	448.1	485.3 (+8.3%)	499.9 (+11.6%)	Correct diagnosis is worth acting on
Budget misallocation	455.9	448.7 (–1.6%)	487.8 (+7.0%)	Correct diagnosis is worth acting on
No real problem	553.0	503.4 (–9.0%)	553.0 (+0.0%)	**Restraint is correctly rewarded**

Deterministic policies only (noop / rules / oracle) — the AI agent is compared separately below, since its results are not yet averaged across multiple seeds.

07 Evidence: closed-loop reasoning from the latest agent run

The screenshot below is the outcome section of the same run shown on the previous page — the agent's own account of what it did, why, what it expected, and what actually happened. Every action it took was later verified against the account's real, measured performance, independently of the model.

```
actions taken:
  #1 p0 set_status(entity_id=cr_broad_a,status=paused) expect as_broad cpa dec 18% within 2p -> confirmed: as_broad cpa: 55.51 -> 42.21 (-24.0%), expected decrease 18%
      rationale: cr_broad_a shows a significant CTR deficit (0.872 vs 1.153, p=0.0) feeding a higher CPC (1.15 vs 0.87) and the worst CPA in the account (75.5 vs sibling 45.64).
  #2 p2 set_budget(entity_id=as_seas_broad,new_daily_budget=35.0) expect as_seas_broad spend dec 19% within 1p -> confirmed: as_seas_broad spend: 301 -> 245 (-18.6%), expected decrease 19%
      rationale: Reduce spend on the account's worst-CPA ad set, which has a statistically significant CVR deficit vs the rest of the account and a monotonic 3-period decline in
  #3 p2 set_budget(entity_id=as_lal,new_daily_budget=113.0) expect as_lal conversions inc 15% within 2p -> confirmed: as_lal conversions: 20.58 -> 25.38 (+23.3%), expected increase 15%
      rationale: Redeploy freed seasonal budget into the account's best-headroom efficient ad set. as_lal has a statistically significant CTR advantage, CPA 32.31 (14% under tar
  #4 p3 set_budget(entity_id=as_seas_gift,new_daily_budget=30.0) expect as_seas_gift spend dec 19% within 1p -> confirmed: as_seas_gift spend: 259 -> 210 (-18.9%), expected decrease 19%
      rationale: Trim spend on the second failing seasonal ad set. as_seas_gift has a CVR that has declined monotonically for four periods, window CPA 57.56 (55% over target) an
```

Every action is checked against reality. Before each action the agent had to state which metric it expected to move, in which direction, by how much, and within how many periods. The platform then measured the actual outcome and marked it `confirmed` independently — the model's own prediction is never taken as fact. All four predictions in this run were later confirmed.

Result: this run against every reference policy, same seed

POLICY	CONVERSIONS	CPA	ACTIONS	REJECTIONS	CONSTRAINT BREACHES
Do-nothing	526.0	38.33	0	—	1
Rule-based baseline	501.0	40.37	8	2	1
AI agent (this run)	**555.0**	**36.56**	4	2	**0**
Oracle (privileged ceiling)	572.0	35.24	2	0	**0**

Single run shown (scenario S1, seed 1) — the most recent controlled test, not yet a multi-seed statistical claim. In this run the agent scored 9.3% above the do-nothing baseline and 20.9% above the deterministic rule-based baseline, met the CPA target that both of those policies missed, and reached 97% of the privileged oracle's score — using only 4 actions and its own investigation, with no access to the correct answer.

08 What this run demonstrates — and how it got there

In this run the agent's own first move was to statistically compare the two creatives inside the account's largest, worst-performing ad set. It found one creative with a significantly worse click-through rate feeding a higher effective cost, paused that single creative rather than cutting the whole ad set's budget, and stated in advance that it expected the ad set's CPA to fall by roughly 18% within two periods. The measured result was a 24% fall, and the run finished with the CPA target met and zero guardrail violations — using targeted, surgical action rather than a broad budget cut.

That result was not the first attempt. Building this correctly required real iteration, which the project's saved run artifacts and version history document directly:

ATTEMPT 1

Same scenario, same seed

The agent investigated the account broadly but never drilled into the specific creative behind the problem, and made several budget changes that its own later measurements confirmed did not work.

→

ROOT-CAUSE ANALYSIS

Instrumented failure review

Replayed the agent's exact reasoning against the underlying data and found it was demanding a statistically stronger signal than the evidence could realistically supply at that sample size — a fixable framing issue, not a data problem.

→

ATTEMPT 2 — SHOWN ABOVE

One prompt change, re-tested

Added one general statistical-reasoning principle to the agent's instructions — no scenario-specific hints. Re-run on the identical scenario and seed: the agent correctly localised and fixed the actual problem.

This is a single documented scenario/seed pair, used here as a concrete, falsifiable example of the engineering process — not a claim that the agent now succeeds on every seed or scenario. Multi-seed validation of this fix is the next planned step.

09 Current status and honest limitations

What this is

- A working prototype under active development, run and evaluated by me directly
- A fully simulated advertising account with hidden, code-defined ground truth
- Runs on a personal Claude subscription via the Claude Code CLI — no paid API key used anywhere in the project
- A deterministic, seeded evaluation harness with 98 automated tests covering the simulator, guardrails, hidden-state isolation, and reproducibility

What this is not (yet)

- Not connected to a real Meta or Google Ads account
- Not a production system, and not commercially deployed
- Not validated across many seeds with the AI policy yet — the closed-loop result shown here is the latest of several documented runs, most recently re-tested after a targeted fix
- Does not guarantee an improvement on any given run — it is measured, and it is sometimes wrong, which is exactly what the evaluation harness is built to catch

10 Engineering practices behind the project

- **Deterministic core, AI at the decision points only.** All arithmetic, statistics, constraint checking and scoring run in plain Python; the language model is used only where judgement is genuinely required — forming and weighing hypotheses, and choosing an action.
- **Reproducibility as a first-class property.** Every run is seeded; the same inputs always produce the same output, across processes and machines.
- **Provider-agnostic by design.** The agent depends on a small backend interface, not on any specific model provider — the system runs and evaluates fully even with no model attached, using the deterministic policies.
- **Zero-cost by constraint, not by accident.** No Anthropic API key, no billing, no paid service is used anywhere in this project; the entire codebase is standard-library Python.

> **In short:** this is a small advertising account, run by code, watched over by a model that has to investigate before it can act, is held to what it predicted, and is checked, at every step, by rules it does not control.